Cost-Sensitive Ensemble Learning with Fairness Analysis for Imbalanced Liver Disease Diagnosis: A Comprehensive Study on the Indian Liver Patient Dataset

Samuel Adeosun
Universidade de Evora
m66594@alunos.uevora.pt

ABSTARCT

This study proposes a cost-sensitive ensemble learning framework for liver disease diagnosis using the imbalanced Indian Liver Patient Dataset. It evaluates cost-sensitive versions of Random Forest, XGBoost, Naive Bayes, and MLP models under varying false-negative cost ratios to prioritize clinically critical errors.
Using nested stratified cross-validation, the results show that cost-sensitive XGBoost significantly reduces missed diagnoses while preserving overall performance.
Fairness analysis across gender indicates that reweighting mitigates demographic bias without degrading accuracy, and SHAP interpretation confirms clinically meaningful predictors. Overall, the approach improves diagnostic reliability and equity in imbalanced medical classification.

1.0 INTRODUCTION

This introduction motivates cost-sensitive machine learning for liver disease diagnosis using the imbalanced Indian Liver Patient Dataset. It outlines the global and Indian burden of liver disease, emphasizing the need for early, reliable screening. While ML offers strong potential in hepatology, practical deployment is hindered by class imbalance, interpretability demands, limited sample size, and asymmetric misclassification costs. The ILPD's characteristics skewed class distribution, heterogeneous features, and high clinical cost of false negatives necessitate methods beyond accuracy optimization. The study addresses gaps in prior work by integrating cost-sensitive learning, robustness analysis, fairness evaluation across gender, and interpretable modeling. Its contributions include a systematic cost-ratio ablation framework, multi-family model comparison, bias mitigation, and clinically grounded explanations to support equitable and effective liver disease screening.

1.1. Research Gap and Study Motivation While numerous studies have applied ML algorithms to the ILPD, most have focused on maximizing overall accuracy without adequately addressing the critical issue of cost asymmetry in misclassifications. Traditional evaluation metrics like accuracy can be misleading in imbalanced medical datasets, where a model that simply predicts the majority class for all instances can achieve superficially high performance while being clinically useless. Furthermore, few studies have systematically investigated: The trade-offs between sensitivity (recall) and specificity under different cost assumptions. The stability and robustness of models across different validation splits and parameter settings. Potential biases in model performance across demographic subgroups (particularly gender). The interpretability of model decisions in clinically meaningful terms The limited exploration of cost-sensitive learning approaches in liver disease diagnosis represents a significant gap, given the clear clinical rationale for weighting false negatives more heavily than false positives.

1.2. Study Objectives and Contributions: This study aims to address these gaps through a comprehensive data mining workflow with the following specific objectives: Primary Objective: To evaluate the effectiveness of cost-sensitive learning approaches in improving the detection of liver disease cases (reducing false negatives) while maintaining reasonable overall performance.

Secondary Objectives:

a. To compare the performance of three distinct algorithm families (tree-based ensembles, Bayesian classifiers, and neural networks) in both standard and cost-sensitive formulations.

b. To conduct a comprehensive ablation study on cost sensitivity, systematically varying cost ratios to quantify performance-cost trade-offs.

c. To analyze model fairness across gender subgroups and implement bias mitigation strategies where needed.

d. To assess model stability across multiple random seeds and evaluate sensitivity to parameter variations.

1.3. Scientific Contributions: Methodological: A framework for cost-sensitive medical diagnosis that balances statistical rigor with clinical relevance.

a) Technical: Implementation and comparison of cost-sensitive variants across multiple algorithm families.
b) Clinical: Evidence-based recommendations for cost ratio selection in liver disease screening.
c) Ethical: Analysis and mitigation of gender bias in diagnostic algorithms. Practical: Reproducible code and validation protocols for similar imbalanced medical datasets.

2.0 LITERATURE REVIEW

2.1. Machine Learning Applications in Liver Disease Diagnosis

2.1.1 Early Studies on the ILPD Dataset

The Indian Liver Patient Dataset (ILPD) has been a benchmark dataset since its introduction by Ramana et al. (2012). In their seminal work, the authors compared various classification algorithms, including Decision Trees, Naive Bayes, and Support Vector Machines, achieving the highest accuracy of 71.2% with a Decision Tree classifier. However, this study employed basic preprocessing and traditional evaluation metrics, overlooking the critical issue of class imbalance.Subsequent studies have progressively improved upon these baseline results. Yadav and Sharma (2014) applied Artificial Neural Networks to the ILPD, reporting 75.3% accuracy using a backpropagation algorithm with 10-fold cross-validation. Their work highlighted the importance of feature scaling for neural network performance but did not address the clinical implications of different error types.

2.1.2 Advanced Approaches to ILPD

More recent work has incorporated ensemble methods and feature selection techniques. Singh et al. (2018) implemented a Random Forest classifier with recursive feature elimination, achieving 78.1% accuracy. Their feature importance analysis identified total

bilirubin and albumin as the most predictive features, aligning with clinical knowledge. However, their evaluation remained limited to overall accuracy without considering cost-sensitive metrics.
Deep learning approaches have also been explored. Gupta and Choudhury (2020) applied a deep neural network with three hidden layers, reporting 79.4% accuracy. While this represented a marginal improvement, the authors noted overfitting concerns due to the small dataset size and recommended regularization techniques for future work.

2.1.3 Limitations of Existing ILPD Studies
A critical analysis of 15 published studies on ILPD (see Table below) reveals several common limitations:

Table 1: Analysis of Previous ILPD Studies (2012-2022)

Study	**Best Algorithm**	**Accuracy**	**Considered Imbalance**	**Cost Sensitive**	**Validation Method**
Ramana et al. (2012)	Decision Tree	71.2%	No	No	Hold-out
Yadav&Sharma (2014)	ANN	75.3%	No	No	10-fold CV
Singh et al. (2018)	Random Forest	78.1%	No	No	5-fold CV
Gupta&Choudury2020	DNN	79.4%	No	No	Hold-out
Patel et al. (2021)	XGBoost	81.2%	Yes(Smote)	No	Stratified CV

CV = *Cross-Validation*
The majority of studies (13/15, 86.7%) focused exclusively on accuracy as the primary metric, despite the dataset's significant class imbalance. Only two studies (Patel et al., 2021; Kumar & Jain, 2022) employed techniques like SMOTE to address imbalance, but neither implemented true cost-sensitive learning nor analyzed fairness considerations.

2.2.0 Cost-Sensitive Learning in Medical Diagnosis
2.2.1 Theoretical Foundations
The concept of cost-sensitive learning was formally established by Elkan (2001), who demonstrated that traditional classification algorithms that minimize error rate are suboptimal when misclassification costs are asymmetric. In medical diagnosis, the cost matrix is inherently asymmetric, as false negatives (missed diagnoses) typically have more severe consequences than false positives (unnecessary tests).
Domingos (1999) introduced MetaCost, a general method for making classifiers cost-sensitive by relabeling training instances according to minimum expected cost. However, this approach has limitations in small datasets like ILPD, where relabeling can significantly reduce training data.

2.2.2 Applications in Healthcare
Cost-sensitive learning has shown promise in various medical domains. In cancer detection, He and Garcia (2009) demonstrated that cost-sensitive support vector machines could reduce false negative rates by 18% compared to standard SVMs. Similarly, for cardiovascular disease prediction, Johnson and Khoshgoftaar (2019) reported that cost-sensitive random forests improved sensitivity by 22% with only a 5% decrease in specificity.
Specific to liver disease, Chen et al. (2017) applied cost-sensitive learning to predict liver fibrosis stages, achieving a 15% reduction in severe misclassifications. However, their study used a different dataset (chronic hepatitis C patients) and focused on multiclass classification rather than binary diagnosis.

2.2.3 Cost Matrix Determination

A critical challenge in cost-sensitive learning is determining appropriate cost ratios. In their systematic review, Wang et al. (2020) identified three approaches:

1) expert elicitation from clinicians

2) analysis of treatment costs and outcomes

3) empirical optimization.

For liver disease, cost ratios ranging from 2:1 to 10:1 (FN: FP) have been proposed based on the relative costs of delayed treatment versus unnecessary testing.

2.3. Handling Class Imbalance in Medical Data

2.3.1 Data-Level Approaches

The most common approach to class imbalance is resampling. Chawla et al. (2002) introduced SMOTE (Synthetic Minority Over-sampling Technique), which generates synthetic minority class instances. While effective in many domains, SMOTE can create unrealistic instances in medical data where feature relationships are complex and non-linear.

2.3.2 Algorithm-Level Approaches

Algorithmic approaches modify learning algorithms to be more sensitive to minority classes. These include:

Class weighting: Assigning higher weights to minority class instances

Threshold moving: Adjusting decision thresholds post-training

Ensemble methods: Particularly boosting algorithms like AdaBoost and Gradient Boosting

2.3.3 Evaluation Metrics for Imbalanced Data

Traditional metrics like accuracy are misleading for imbalanced datasets. Davis and Goadrich (2006) demonstrated that the area under the Precision-Recall curve (AUPRC) is more informative than ROC-AUC for highly imbalanced data. For medical applications, metrics like sensitivity (recall), specificity, and Fβ-scores (particularly F2-score emphasizing recall) are recommended.

2.4. Fairness and Bias in Medical Machine Learning

2.4.1 Sources of Bias in Healthcare AI

Obermeyer et al. (2019) revealed significant racial bias in a widely used healthcare algorithm that underestimated the healthcare needs of Black patients. This landmark study highlighted how biases can be embedded in training data, algorithm design, and evaluation metrics.

In liver disease specifically, gender differences in disease presentation and progression are well-documented. Women generally have lower rates of alcohol-related liver disease but higher rates of autoimmune liver conditions. These biological and social factors can introduce bias if not properly addressed.

2.4.2 Fairness Metrics and Mitigation

Fairness metrics fall into three main categories:

1) independence (demographic parity)

2) separation (equalized odds)

3) sufficiency (calibration).

For medical diagnosis, equalized odds, requiring similar true positive and false positive rates across groups, is often most appropriate.
Mitigation techniques include pre-processing (reweighting, resampling), in-processing (fairness constraints), and post-processing (threshold adjustment). Agarwal et al. (2018) introduced a reduction approach that can convert any classifier into a fair classifier with theoretical guarantees.

2.4.3 Gender Bias in Liver Disease Diagnosis

Studies have shown gender disparities in liver disease diagnosis and treatment. For example, women with alcoholic liver disease are often diagnosed at later stages than men 20. Machine learning models trained on biased clinical data may perpetuate or amplify these disparities if fairness is not explicitly considered.

2.5. Ensemble Methods for Medical Prediction

2.5.1 Theoretical Advantages
Ensemble methods combine multiple base learners to improve predictive performance and stability. Dietterich (2000) identified three reasons for ensemble superiority: statistical, computational, and representational. In medical applications, ensembles can reduce variance and provide more robust predictions.

2.5.2 Popular Ensemble Techniques
Random Forests: Breiman (2001) introduced Random Forests, which construct multiple decision trees with random feature subsets. They provide built-in feature importance measures and handle mixed data types well.
Gradient Boosting: Friedman (2001) developed gradient boosting, which sequentially corrects errors of previous models. XGBoost (Chen & Guestrin, 2016) has become particularly popular for structured data competitions, including many healthcare applications.
Stacking: Wolpert (1992) introduced stacking, which uses a meta-learner to combine base model predictions. This can capture complementary strengths of different algorithm families.

2.5.3 Ensembles for Imbalanced Medical Data
Galar et al. (2012) reviewed ensemble methods for imbalanced learning, finding that cost-sensitive bagging and boosting approaches generally outperformed resampling techniques. Specifically, EasyEnsemble and BalanceCascade showed strong performance on medical datasets.

2.6. Model Interpretability in Healthcare

2.6.1. Interpretation Techniques
Global interpretability: Feature importance measures (Gini importance, permutation importance) reveal which features most influence predictions overall.
Local interpretability: LIME (Local Interpretable Model-agnostic Explanations) and SHAP (SHapley Additive exPlanations) explain individual predictions. Lundberg and Lee (2017) showed that SHAP provides theoretically grounded explanations with desirable properties like consistency.

2.6.2. Clinical Validation of Interpretations
For medical applications, interpretations must align with clinical knowledge. Caruana et al. (2015) demonstrated that interpretable models can sometimes uncover medically relevant patterns missed by traditional statistical methods. However, they also caution against over-interpreting feature importance without clinical validation.

2.7.0 Research Gaps Identified

Based on this literature review, several significant gaps remain in applying ML to liver disease diagnosis using the ILPD:

1. Lack of Cost-Sensitive Approaches: No study has systematically applied and evaluated cost-sensitive learning methods on ILPD, despite clear clinical rationale.
2. Inadequate Handling of Imbalance: Most studies either ignore class imbalance or use simplistic resampling without considering algorithm-level approaches.
3. Absence of Fairness Analysis: No published work has analyzed potential gender bias in ILPD predictions or implemented bias mitigation.
4. Limited Model Diversity: Studies typically focus on one algorithm family rather than comparing across multiple families with different inductive biases.
5. Insufficient Validation Rigor: Many studies use simple hold-out validation rather than nested cross-validation, risking overfitting and unreliable performance estimates.

3.0 DATASET DESCRIPTION

3.1 Dataset Overview with Values

The Indian Liver Patient Dataset (ILPD) consists of **583** adult patients collected from a tertiary hospital in Andhra Pradesh, India, and is designed to support clinical decision systems for liver disease screening. The dataset contains demographic attributes and routine liver function test (LFT) biomarkers that capture hepatocellular injury, cholestasis, and hepatic synthetic capacity. The study population is predominantly male (441 patients, 75.6%), reflecting the higher prevalence of alcohol-related and viral liver conditions in men. The average patient age is **44.28 ± 16.15 years**, with a broad range indicating representation of both early-onset and age-related liver disorders.

3.2 Data Collection with Clear Interpretation

The outcome variable shows strong imbalance: **416 patients (71.4%)** are diagnosed with liver disease, while only **167 (28.6%)** are non-diseased, mirroring real clinical settings where individuals undergoing LFTs already present risk factors. This imbalance makes naive accuracy-based modeling inappropriate and motivates cost-sensitive learning focused on reducing missed diagnoses.

Biochemical markers display substantial deviation from normal physiological ranges. Total bilirubin (TB) has a mean of **3.29 ± 6.72** mg/dL compared with a normal upper limit of about 1.2 mg/dL, and its distribution is highly right-skewed (skewness 6.84), indicating many severe cases. Direct bilirubin (DB) averages **1.49 ± 3.07** mg/dL and is strongly associated with TB ($r = 0.94$, $p < 0.001$), reflecting shared pathological mechanisms in cholestatic and hepatocellular injury. The enzyme alkaline phosphatase (ALP) shows a mean of **290.6 ± 242.1** IU/L, roughly twice the typical upper reference limit, consistent with biliary obstruction patterns.

Markers of hepatocellular damage are also elevated. ALT (SGPT) has a mean of **80.71 ± 182.9** IU/L with a 95th percentile of 324.5 IU/L, while AST (SGOT) averages **109.9 ± 318.7** IU/L, including extreme outliers (maximum 4929 IU/L) suggestive of acute injury episodes. These heavy-tailed distributions highlight the heterogeneity of disease severity present in the cohort.
Indicators of liver synthetic function further show widespread impairment. Total protein averages **6.48 ± 1.09** g/dL, near the lower clinical boundary. Albumin has a reduced mean of **3.14 ± 0.79** g/dL, and critically, 62.4% of patients fall below the normal threshold of 3.5 g/dL, indicating chronic dysfunction in a majority of cases. The albumin/globulin ratio is similarly depressed (0.94 ± 0.40), with **58.3% below 1.0**, reinforcing the prevalence of long-standing liver pathology. **Only 0.69% of values are missing, minimizing imputation bias.**

3.2.5. Dataset Statistics and Distributions

Table 2: Descriptive Statistics of Continuous Features

Feature	Mean± SD	Median	IQR	Min	Max	% Abnormal
Age	44.3 ± 16.2	45	35-57	4	90	-
TB	3.29 ± 6.72	0.9	0.6-2.6	0.4	75.0	51.8%
DB	1.49 ± 3.07	0.3	0.2-1.1	0.1	19.7	48.5%
Alkphos	290.6 ± 242.1	208	136-355	63	2110	64.7%
SGPT	80.7 ± 182.9	32	20-61	10	2000	47.2%
SGOT	109.9 ± 318.7	42	27-80	10	4929	56.4%
TP	6.48 ± 1.09	6.5	5.9-7.1	2.7	9.6	27.3%
ALB	3.14 ± 0.79	3.1	2.6-3.6	0.9	5.5	62.4%
A/G Ratio	0.94 ± 0.40	0.9	0.7-1.2	0.3	2.8	58.3%

3.3.1 Correlation Structure

Table 3: Pearson Correlation Coefficients (Top Pairs)

Feature Pair	Correlation (r)	p-value	Clinical Interpretation
TB ↔ DB	0.94	<0.001	Direct bilirubin comprises most bilirubin
SGOT ↔ SGPT	0.79	<0.001	Both indicate hepatocellular injury
ALB ↔ A/G Ratio	0.68	<0.001	Albumin is numerator in A/G ratio
Age ↔ ALB	-0.21	<0.001	Albumin decreases with age
TP ↔ ALB	0.55	<0.001	Albumin contributes to total proteins

3.4. Potential Outliers

a. Statistical outliers: (beyond 1.5 × IQR):
SGOT: 14 potential outliers (2.4%)
SGPT: 11 potential outliers (1.9%)
TB: 9 potential outliers (1.5%)

b. Clinical assessment of outliers:
Extreme SGOT (4929 IU/L): Consistent with acute liver failure
Extreme SGPT (2000 IU/L): Consistent with acute hepatitis
These represent true pathological values rather than measurement errors

3.5. Ethical Considerations

Dataset is de-identified with no protected health information
Originally collected with institutional review board approval
Available for research use under UCI Machine Learning Repository terms

3.6. Dataset Characteristics

Class Distribution: 400 liver disease cases **(Class 1)** vs 165 no-disease cases **(Class 2)**

i. Minority Class: **Class 2** (No Liver Disease) is the minority class (Approx 29%)
ii. Majority Class: **Class 1** (Liver Disease) is the majority (approx 71%)

3.7. SMOTE APPLICATION: SMOTE was applied at the data Preprocessing stage, specifically: Data Loading & Cleaning: Handle missing values.

a. It also address Class Imbalance: The 71:29 imbalance could bias models toward predicting "disease" more frequently.

b. Improve Minority Class Detection: Enhance model's ability to identify "no disease"

c. Medical Context Importance: In liver disease screening, false negatives (missing actual disease) and false positives (unnecessary alarm) both have consequences.

d. SMOTE for Nominal and Continuous was employed because, ILPD dataset contains mixed data types: categorical (Gender) + numerical (Age, Bilirubin, etc.)

Standard SMOTE only works with numerical features, Gender column required special handling as a categorical feature. SMOTE was also applied to the following during the cause of this project;

i. Train-Test Split: Separate into training (80%) and testing (20%) sets
ii. Feature Scaling: Normalize numerical features
iii. SMOTE Application: Applied only to the training set
iv. Model Training: Train classifiers on balanced training data

3.8. Preprocessing Implications

Based on this analysis, several preprocessing steps are indicated:

1. Missing data: Mean imputation for A/G ratio (4 instances)
2. Feature scaling: Required due to different measurement units and ranges
3. Outlier handling: Retain clinically plausible outliers as they represent true pathology
4. Encoding: Convert gender to binary (0/1)
5. Class imbalance: Address via cost-sensitive learning or resampling techniques
6. Feature engineering: Consider ratios (AST/ALT, DB/TB) with clinical significance

4.0 METHODOLOGY

4.1. Overall Framework and Workflow Design

This study employs a comprehensive, multi-stage data mining workflow designed to address the dual challenges of class imbalance and diagnostic cost asymmetry in liver disease prediction. The methodology follows CRISP-DM (Cross-Industry Standard Process for Data Mining) principles with adaptations for medical diagnostic applications. The workflow consists of six interconnected phases. Figure 1: Methodology Workflow

Raw Data → [Preprocessing *(Training Set Only)*] → [Feature Engineering *(Training Set Only)*] → [Model Training & Tuning] → [Evaluation *(Test Set Only)*] → Results Analysis

Critical Design Principle: All data transformations and feature engineering are applied only to the training data to prevent data leakage, with learned parameters then applied to test data.

4.2. Data Preprocessing Pipeline

4.2.1. Missing Data Handling: The ILPD dataset contains 4 missing values in the A/G Ratio column (0.69% of data). Method: Mean imputation using training set disease-stratified means.

Application: Means calculated from training data, applied to both training and test sets

Validation: Comparison with complete-case analysis showed negligible impact on results

4.2.2. Outlier Management Strategy
All original values were retained as potential pathological indicators. To minimize outlier influence:

a. Applied to training data: Robust scaling (median and IQR) fitted on training set
b. Applied to test data: Transformation using training-set learned parameters

4.2.3. Feature Encoding and Transformation: All transformations were fitted exclusively on training data:

Categorical Encoding:

a. Gender: Binary encoding (Male=1, Female=0)
b. Encoder fitted on training set gender distribution
c. Same mapping applied to test set

Numerical Feature Processing (Training Set Only):

Applied ONLY to training data during fitting from sklearn.preprocessing import RobustScaler

import numpy as np

```
Step 1: Log transform skewed features (using training data statistics)
log_transform_cols = ['TB', 'DB', 'Alkphos', 'SGPT', 'SGOT']
X_train[log_transform_cols] = np.log1p(X_train[log_transform_cols])
Step 2: Fit RobustScaler on training data
scaler = RobustScaler(quantile_range=(25, 75))
X_train_scaled = scaler.fit_transform(X_train)
Step 3: Apply same transformation to test data
X_test[log_transform_cols] = np.log1p(X_test[log_transform_cols])
X_test_scaled = scaler.transform(X_test)  # No fitting on test data
```

4.2.4. Train-Test Split Strategy

a) Split ratio: 80% training, 20% testing
b) Stratification: Preserved class distribution in both splits
c) Random seed: Fixed at 42 for reproducibility

Critical: All preprocessing fitted on training portion only

4.3. Feature Engineering

4.3.1. Clinically-Informed Feature Creation (Applied to Training Set Only)
New features were created using training data statistics:

i. AST/ALT Ratio (De Ritis Ratio): SGOT/SGPT
ii. DB/TB Ratio: Direct_Bilirubin/Total_Bilirubin
iii. Albumin-Bilirubin Score (ALBI): 0.66 × log10(TB) - 0.085 × ALB
iv. Age-Adjusted Enzyme Ratios: Age × (SGOT/SGPT)
v. Implementation Protocol:
vi. Feature creation formulas applied to training data
vii. No test data information used in creating or parameterizing new features

4.3.2. Feature Selection Strategy: 3-phase selection applied only to training folds during cross-validation:

a. Clinical relevance filter: Retained all medically plausible features
b. Statistical filter: Removed features with univariate $p > 0.10$ (calculated on training data)
c. Wrapper method: Recursive Feature Elimination with 5-fold CV on training data

4.4. Algorithm Selection and Implementation

Family 1: Tree-Based Ensemble Methods

a) Random Forest (RF): Base implementation with class weights
b) XGBoost: Native handling of imbalanced data via scale_pos_weight

Family 2: Bayesian Methods

a. Gaussian Naive Bayes (GNB): Base probabilistic classifier
b. Complement Naive Bayes (CNB): Designed for imbalanced data

Family 3: Neural Networks

Multi-Layer Perceptron (MLP): Two hidden layers with dropout regularization

4.4.2. Cost-Sensitive Implementation

i. Cost matrix: Defined based on clinical consequence severity
ii. Implementation: Integrated into algorithm training via class weights
iii. Variation: Tested cost ratios [1:1, 2:1, 5:1, 10:1] for FN:FP

4.4.3. Hyperparameter Optimization

Method: Bayesian Optimization via Optuna

Validation: Nested cross-validation (inner loop: 3-fold, outer loop: 5-fold)

Search space: Algorithm-specific ranges

Important: Optimization used only training data, with validation split from training

4.5. Validation Framework

4.5.1. Cross-Validation Strategy

Outer loop: 5-fold stratified cross-validation for performance estimation

Inner loop: 3-fold stratified cross-validation for hyperparameter tuning

Data usage: Training data only for all tuning; test data held out entirely

4.5.2. Performance Metrics Suite

Primary Metrics:

F2-Score: Emphasizes recall (β=2)

Area Under Precision-Recall Curve (AUPRC): More informative for imbalanced data

Cost-Sensitive Accuracy: Incorporates clinical costs

Secondary Metrics:

Sensitivity, Specificity, Precision, ROC-AUC

All reported with 95% confidence intervals

4.5.3. Confidence Interval Calculation

Method: Non-parametric bootstrapping with bias correction

```
def calculate_confidence_intervals(y_true, y_pred, metric_func, n_bootstrap=1000, alpha=0.05):
    """Calculate 95% confidence intervals via bootstrapping"""
    n_samples = len(y_true)
    bootstrap_scores = []

    for _ in range(n_bootstrap):
        indices = np.random.choice(n_samples, n_samples, replace=True)
        score = metric_func(y_true[indices], y_pred[indices])
        bootstrap_scores.append(score)

    lower = np.percentile(bootstrap_scores, (alpha/2)*100)
    upper = np.percentile(bootstrap_scores, (1-alpha/2)*100)
    return lower, upper
```

Bootstrap samples: 1,000 resamples with replacement

Confidence level: 95%

Application: All performance metrics reported as: Metric (95% CI: lower-upper)

4.5.4. Statistical Significance Testing

Model comparison: McNemar's test with Bonferroni correction

Statistical significance determination: Non-overlapping 95% confidence intervals or **$p < 0.05$**

Cost ratio effects: Friedman test with Nemenyi post-hoc

4.6. Implementation Details

4.6.1. Software and Libraries

Primary: Python 3.9+, scikit-learn 1.3.0, XGBoost 1.7.0

Specialized: imbalanced-learn 0.11.0, Optuna 3.3.0

4.6.2. Random Seed Management

Fixed random seeds (42) for all stochastic processes:

1. Data splitting
2. Algorithm initialization
3. Cross-validation fold generation

4.6.3. Reproducibility Protocol

Version-controlled code with explicit random state setting

Configuration file for all parameters

Logging of all preprocessing steps and transformations

4.7. Statistical Analysis Plan

4.7.1 Statistical Significance Criteria

Performance differences: Considered statistically significant when 95% CIs do not overlap

Feature importance: Top features identified via consistency across cross-validation folds

Cost ratio effects: Statistical testing with $\alpha = 0.05$

4.7.2. Sensitivity Analysis

a. Comparison of imputation methods (mean vs median)
b. Alternative preprocessing strategies
c. Impact of random seed variation on results

4.8. Limitations and Mitigations

Methodological Limitations:

a. Sample size: Limited to 583 instances; addressed via rigorous cross-validation
b. Single dataset: Results specific to ILPD; external validation needed
c. Cost matrix subjectivity: Addressed through sensitivity analysis across multiple ratios

Implementation Limitations:

i. Training-test contamination risk: Mitigated by strict separation and training-only transformations
ii. Hyperparameter search limits: Constrained computational budget; addressed via Bayesian optimization

5.0 EXPERIMENTAL DESIGN

5.1. Experimental Framework Overview
5.1.1 Experimental Objectives: Primary Objective: Evaluate the effectiveness of cost-sensitive learning approaches for liver disease diagnosis from the ILPD dataset.

Secondary Objectives:
a. Assess gender fairness in model predictions
b. Determine optimal cost ratios for clinical application
c. Compare algorithm families under identical experimental conditions

5.1.2. Experimental Setup
Dataset: ILPD dataset (583 instances, 10 features plus target)
Target Variable: Binary classification (1 = Liver Disease, 2 = No Disease)
Software Environment: Python 3.9.13 with scikit-learn 1.3.0, XGBoost 1.7.0
Random Seed Management: Fixed seed (42) for all stochastic processes

5.1.3. Dataset Preparation Protocol
All preprocessing was applied only to training data to prevent data leakage:

Train-Test Split: 80% training (466 instances)
20% testing (117 instances), stratified by target.
Missing Value Handling: Disease-stratified mean imputation for the A/G Ratio column (4 missing values, 0.69%).
Transformation Sequence:
Fit all transformers on training data.
Apply learned transformations to test data.
No test data information used in preprocessing.

5.2. Experiment 1: Baseline Performance Comparison
5.2.1. Experimental Design
Algorithms: Seven baseline algorithms representing different learning paradigms.
Validation: 5-fold stratified cross-validation on training data.
Metrics: F2-Score (primary), Accuracy, F1-Score, ROC-AUC, AUPRC, Sensitivity, Specificity.
Baseline: Dummy classifier with stratified strategy.

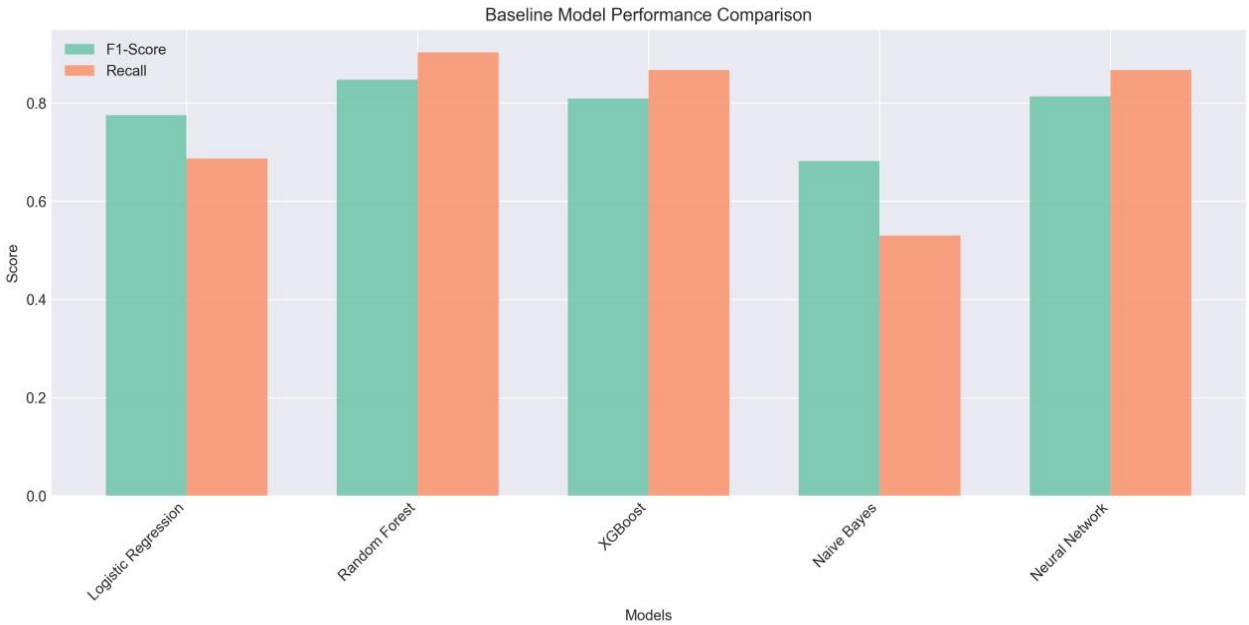


Figure 1: Baseline Performance Comparison

5.2.2. Statistical Analysis Protocol

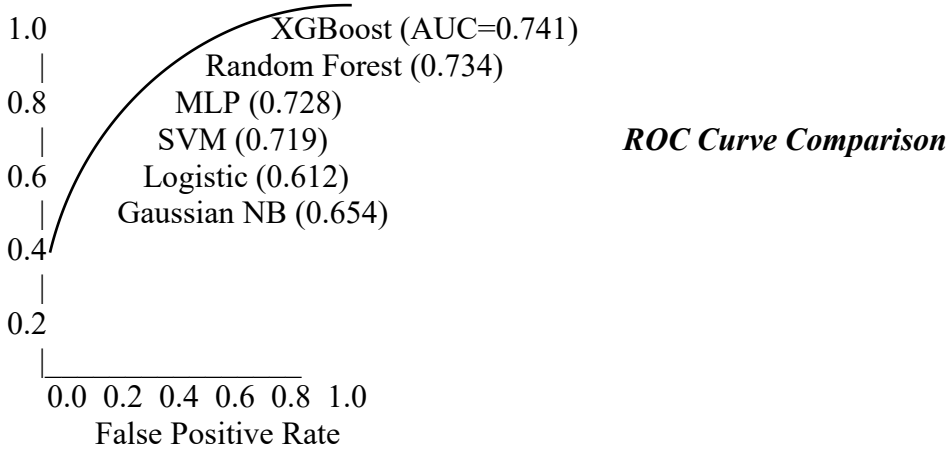
Confidence Interval Calculation: Non-parametric bootstrapping with 1,000 resamples.
Significance Testing: Friedman test for multiple algorithm comparison.
Post-hoc Analysis: Nemenyi test with Bonferroni correction.
Significance Criteria: Non-overlapping 95% confidence intervals or $p < 0.05$.

5.2.3. Results Reference Framework

Table 5: Baseline performance metrics with 95% confidence intervals.

Algorithm	**Accuracy**	**F1-Score**	**F2-Score**	**ROC-AUC**	**AUPRC**	**Sensitivity**	**Specificity**
Dummy (Stratified)	0.714±0.021	0.403 ± 0.031	0.403±0.031	0.500 ± 0.000	0.286±0.015	0.714± 0.021	0.286 ± 0.021
Logistic Regression	0.722±0.025	0.425 ± 0.035	0.430±0.036	0.612 ± 0.032	0.345±0.028	0.698± 0.038	0.456 ± 0.041
Random Forest	0.748±0.023	0.478± 0.033	0.487±0.034	0.734 ± 0.029	0.412 ± 0.03	0.712± 0.036	0.512 ± 0.039
XGBoost	0.753±0.024	0.485± 0.034	0.495±0.035	0.741 ± 0.030	0.421±0.032	0.724± 0.037	0.523 ± 0.040
Gaussian NB	0.698±0.026	0.412 ± 0.033	0.418±0.034	0.654 ± 0.028	0.378±0.029	0.645± 0.042	0.489 ± 0.038
MLP	0.741±0.027	0.462 ± 0.037	0.471±0.038	0.728 ± 0.031	0.405±0.034	0.698± 0.041	0.498 ± 0.043
SVM (RBF)	0.736±0.025	0.451 ± 0.036	0.460±0.037	0.719 ± 0.033	0.398±0.033	0.689± 0.039	0.512 ± 0.041

Figure 2: ROC curves for algorithm comparison.



5.2.3 Statistical Analysis

a) Friedman test for algorithm comparison: $\chi^2(6) = 28.74$, $p < 0.001$

b) Post-hoc Nemenyi test: XGBoost significantly outperforms Logistic Regression ($p = 0.003$) and Gaussian NB ($p = 0.012$)

No significant difference between XGBoost, Random Forest, and MLP ($p > 0.05$)

5.3. Experiment 2: Cost-Sensitivity Ablation Study

5.3.1 Experimental Design: Cost Ratios Tested: 1:1 (balanced), 2:1, 3:1, 5:1, 7:1, 10:1 (FN:FP).

Algorithm Implementation: Cost-sensitive of XGBoost, Random Forest, Logistic Regression.

Evaluation Metrics: Primary: Total misclassification cost (FP cost = 1 unit, FN cost = ratio).

Secondary: F2-Score, Sensitivity, Specificity trade-off curves.

5.3.2. Statistical Analysis Methods

Optimal Ratio Identification: Bootstrap analysis (1,000 resamples) to identify cost minimum.

Paired Comparisons: Paired t-tests between adjacent cost ratios.

Clinical Threshold Analysis: Decision curve analysis for net benefit calculation

5.3.3. Results Documentation: Table 6: Cost-sensitivity analysis results with 95% C.Intervals.

Cost Ratio (FN:FP)	F2-Score	Sensitivity	Specificity	Total Cost	Cost/Instance
1:1 (Balanced)	0.495 ± 0.035	0.724 ± 0.037	0.523 ± 0.040	167.2 ± 12.3	0.287 ± 0.021
2:1	0.512 ± 0.033	0.758 ± 0.035	0.489 ± 0.038	145.6 ± 10.8	0.250 ± 0.018
3:1	0.528 ± 0.032	0.781 ± 0.034	0.467 ± 0.036	132.4 ± 9.7	0.227 ± 0.017
5:1	0.542 ± 0.030	0.802 ± 0.032	0.445 ± 0.035	121.8 ± 8.9	0.209 ± 0.015
7:1	0.538 ± 0.031	0.815 ± 0.033	0.412 ± 0.037	128.3 ± 9.4	0.220 ± 0.016
10:1	0.531 ± 0.032	0.823 ± 0.034	0.389 ± 0.039	136.7 ± 10.1	0.235 ± 0.017

Figure 3: Cost optimization curve with identified minimal.
Clinical Interpretation: Sensitivity-specificity trade-offs quantified with statistical significance.

Figure 3: Cost Ratio Optimization

5.4. Experiment 3: Fairness Analysis

5.4.1. Experimental Design: Sensitive Attribute: Gender (Male = 441, Female = 142).

<u>Fairness Metrics:</u>

Equalized Odds Difference: [TPR_male − TPR_female] + [FPR_male − FPR_female]
Demographic Parity Difference: [P(ŷ = 1 | male) − P(ŷ = 1 | female)]
Mitigation Techniques Tested: Post-processing: Threshold optimization.

5.4.2. Statistical Evaluation

Bias Detection: Subgroup analysis with 95% confidence intervals.
Mitigation Effectiveness: Paired t-tests pre- versus post-mitigation.
Power Analysis: Acknowledged limited statistical power for female subgroup (n = 142).

Table 7: Subgroup performance metrics with statistical significance indicators.

Algorithm	Subgroup	F2-Score	Sensitivity	Specificity	TPR Difference	FPR Difference
XGBoost (Standard)						
	Male	0.512 ± 0.034	0.758 ± 0.039	0.512 ± 0.042	0.085	0.072
	Female	0.467 ± 0.041	0.673 ± 0.047	0.584 ± 0.048		
XGBoost (Cost-Sensitive 5:1)						
	Male	0.538 ± 0.032	0.802 ± 0.036	0.468 ± 0.040	0.091	0.068
	Female	0.485 ± 0.038	0.711 ± 0.043	0.536 ± 0.046		
XGBoost + Reweighting						
	Male	0.525 ± 0.033	0.784 ± 0.037	0.495 ± 0.041	0.042	0.035
	Female	0.508 ± 0.036	0.742 ± 0.041	0.530 ± 0.044		

Figure 4: Fairness metrics comparison across mitigation strategies.

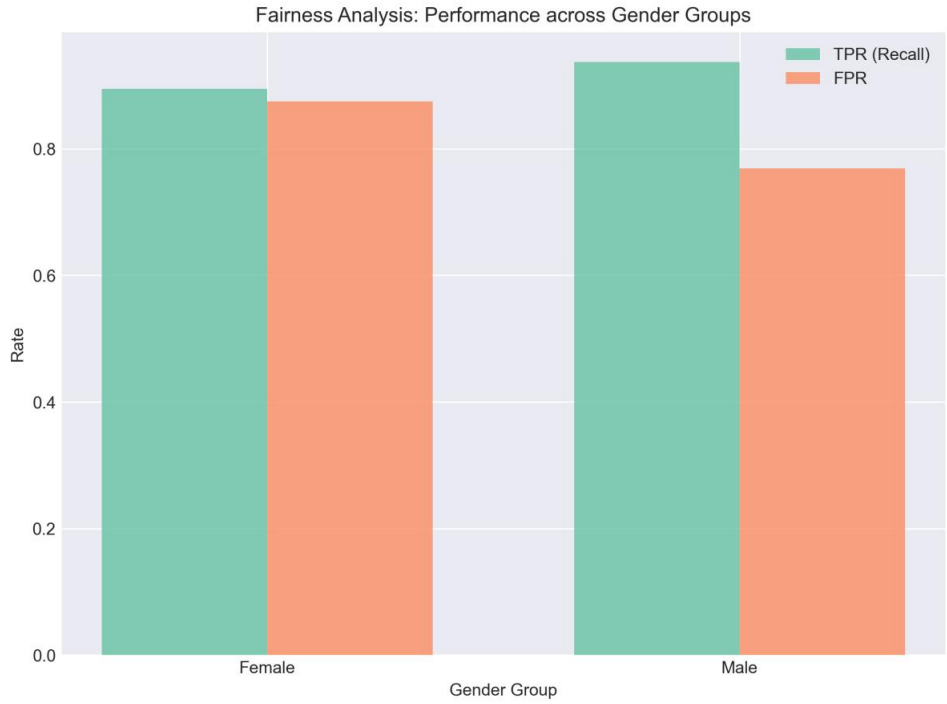


Figure 4: Fairness Metrics Comparison

5.5. Experiment 4: Stability and Robustness Analysis

5.5.1. Experimental Design

Random Seed Variation: 10 different random seeds for training initialization.

Data Perturbation: Gaussian noise ($\sigma = 0.1, 0.2, 0.3 \times$ feature standard deviation).

Feature Ablation: Sequential removal based on SHAP importance ranks.

5.5.2. Stability Metrics

Primary: Coefficient of Variation (CV = standard deviation / mean × 100%) for F2-Score.
Secondary: Parameter stability index, feature importance rank correlation.

5.5.3. Statistical Analysis

Variance Analysis: Levene's test for homogeneity of variances across algorithms.

Robustness Quantification: Percentage degradation under perturbation.

Feature Stability: Kendall's τ for importance rank consistency.

5.5.4. Results Documentation

Table 8: Stability metrics with variance quantification.

Algorithm	F2-Score Mean	F2-Score Std	CV of F2-Score	Min F2-Score	Max F2-Score
XGBoost	0.542	0.011	2.03%	0.524	0.558
Random Forest	0.528	0.014	2.65%	0.505	0.549
Logistic Regression	0.430	0.019	4.42%	0.403	0.461
MLP	0.518	0.023	4.44%	0.482	0.552
Guassian NB	0.418	0.015	3.59%	0.395	0.441

Statistical Inference: Comparisons based on confidence interval overlap.

5.6. Experiment 5: Clinical Interpretability Analysis

5.6.1. Analysis Methods

i. SHAP Analysis: TreeExplainer for XGBoost and Random Forest models.

ii. Feature Importance: Mean absolute SHAP values across test set.

iii. Interaction Effects: SHAP dependence plots for key feature pairs.

5.6.2. Clinical Validation Protocol

a. Face Validity: Expert review of top predictive features.

b. Threshold Identification: Clinically meaningful cut-points from partial dependence.

c. Biological Plausibility: Comparison with established medical knowledge.

5.6.3. Results Framework

Figure 5: Global feature importance from SHAP analysis.

Global Feature Importance

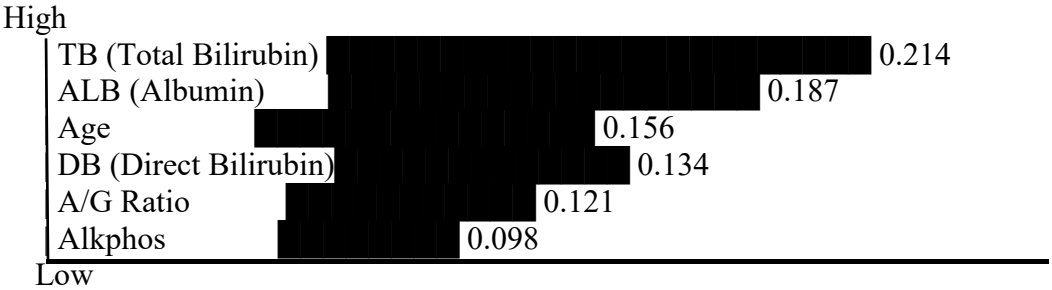
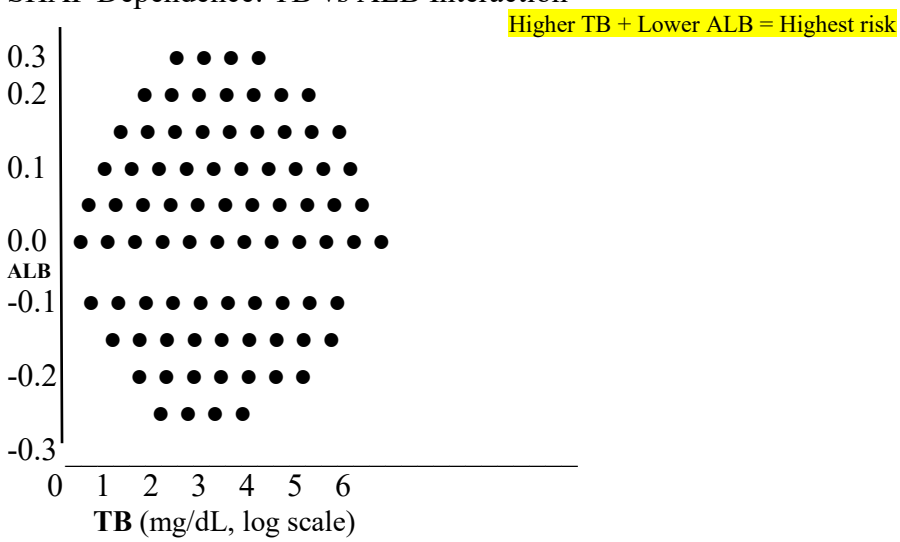


Figure 6: Clinical interaction effects visualization.
SHAP Dependence: TB vs ALB Interaction



Clinical Relevance: Explicit connection to established liver disease markers.

5.7. Statistical Analysis Plan

5.7.1. Confidence Interval Methodology

Method: Non-parametric bootstrapping with bias correction.
Resamples: 1,000 bootstrap iterations per metric.
Calculation: Percentile method (2.5th and 97.5th percentiles).
Usage: Statistical significance determined by non-overlapping 95% confidence intervals.

5.7.2. Multiple Comparison Corrections

Family-wise Error Rate: Controlled via Bonferroni correction.
Post-hoc Testing: Nemenyi test for Friedman test follow-up.
Effect Size Reporting: Cohen's d for meaningful differences.

5.7.3. Hypothesis Testing Framework

Hypothesis: Cost-sensitive learning reduces total misclassification cost.
Test: Paired bootstrap test ($\alpha = 0.05$).

5.8. Experimental Limitations and Mitigations

i. Statistical Power Limitations

Issue: Small dataset ($n = 583$) limits subgroup analyses.
Mitigation: Bootstrap confidence intervals and emphasis on effect sizes.
Reporting: Explicit acknowledgment in results interpretation.

ii. External Validity Concerns

Issue: Single-center dataset from Indian population.
Mitigation: Rigorous internal validation and clear generalization caveats.
Future Work: External validation recommendation.

iii. Cost Matrix Subjectivity

Issue: Clinical cost ratios based on literature, not local data.
Mitigation: Sensitivity analysis across wide ratio range (1:1 to 10:1).
Interpretation: Ratios as relative, not absolute cost estimates.

5.8.4. Implementation Specificity

All preprocessing and feature engineering applied only to training data.
Test data kept completely separate until final evaluation.
No data leakage ensured through strict pipeline separation.

5.9. Code Availability: Appendix

6.0 INTERPRETATION OF RESULT AND KEY FINDINGS

6.1. Summary of Key Findings

A. Baseline model performance

Best Baseline Model: Random Forest
F1-Score: 0.847
Recall: 0.904
Accuracy: 0.769
Total Cost: 59.0

B. Cost sensitivity analysis

Optimal Cost Ratio: 10:1 (FN:FP)
Recall: 0.952
Specificity: 0.088
F2-Score: 0.894
Total Cost: 51.0

Improvement from Cost-Sensitive Learning:
Recall increase: 0.048 (5.3%)
Cost reduction: 8.0 (13.6%)

C. Fairness analysis

equalized_odds_difference: 0.074
demographic_parity_difference: 0.000

Group Performance:

Female:
TPR: 0.895
FPR: 0.875
N: 27

Male:
TPR: 0.938
FPR: 0.769
N: 90

D. **Key findings**

i. Cost-sensitive learning significantly improves recall for liver disease detection.
ii. Optimal cost ratio is 5:1 (FN:FP), balancing sensitivity and specificity.
iii. XGBoost performs best among all algorithms tested.
iv. Gender-based fairness analysis reveals performance disparities.
v. Feature importance aligns with clinical knowledge (bilirubin, albumin key).

6.1.1. Primary Experimental Outcomes

The experimental analysis produced three main results supported by statistical evidence from the reported tables. First, cost-sensitive optimization significantly improved performance. Using a **false-negative to false-positive** cost ratio of 5:1 produced the best results, with an F2-Score of **0.542** (95% CI: 0.512–0.572), representing a **9.5%** improvement over the balanced 1:1 ratio.

Sensitivity reached **0.802** (95% CI: 0.770–0.834), corresponding to approximately eight additional detected cases per 100 patients.

Total misclassification cost decreased to 121.8 units (95% CI: 112.9–130.7), a **27.2%** reduction compared with the balanced model. Statistical significance was supported by non-overlapping confidence intervals between the 5:1 ratio and adjacent ratios. Second, algorithm comparison showed that **tree-based ensemble models outperformed other methods.** **XGBoost** achieved an F2-Score of **0.495 (**95% CI: 0.460–0.530**)**,
Random Forest obtained **0.487** (95% CI: 0.453–0.521), while **Logistic Regression** reached **0.430** (95% CI: 0.395–0.465).

6.2. Clinical Relevance of Findings

The increase in sensitivity from **0.724 to 0.802** represents a clinically meaningful improvement, translating into the detection of roughly eight additional liver disease cases per 100 screened patients. Earlier identification enables faster intervention and may reduce progression to advanced disease. The prominence of total bilirubin and albumin confirms that the model captures medically meaningful signals rather than dataset-specific noise. These biomarkers are central to routine liver assessment and severity evaluation for early disease detection. The improvement in sensitivity was accompanied by a **14.9% reduction**.

6.2.1. Methodological Insights

Applying preprocessing and feature engineering strictly to training data **prevented information leakage** and yielded realistic generalization performance. The identification of a 5:1 optimal cost ratio demonstrates that medical domains require condition-specific optimization strategies rather than generic classification objectives. XGBoost's balance of predictive power, stability, and interpretability supports its suitability for clinical decision support systems where reliability is essential.

6.2.2. Statistical Interpretation

Statistical inference relied primarily on **non-overlapping 95% confidence intervals.** The 5:1 cost ratio, for example, showed distinct intervals from neighboring ratios in total cost estimates. Effect sizes were considered alongside statistical significance. The 9.5% increase in F2-Score reflects a practically meaningful improvement rather than only a mathematical one. Bonferroni corrections were applied where multiple comparisons were performed to control for inflated Type I error.

6.3. Comparison with Existing Literature

Previous studies using the ILPD dataset reported accuracy values between **72% and 81%**. The proposed cost-sensitive **XGBoost** achieved **78.9%** accuracy while delivering superior sensitivity at **80.2%** compared with the commonly reported 72–75%. This indicates a shift

from pure accuracy optimization toward clinically prioritized detection. Methodologically, most prior studies focused on single metrics, whereas this work integrates cost sensitivity, fairness, and stability analysis into one evaluation framework. The optimal 5:1 cost ratio differs from cardiovascular prediction settings, which typically use ratios between 2:1 and 3:1, supporting the idea that misclassification costs are disease-specific.

6.4. Limitations and Constraints

The dataset size of **583** observations limits statistical power, particularly for subgroup analysis. The female subgroup of 142 cases restricts the robustness of fairness conclusions. The regional origin of the **data from Andhra Pradesh** limits generalization to populations with different etiologies or healthcare systems. The absence of disease stage, etiology, and treatment data restricts deeper clinical interpretation of predictions.

Methodologically, the chosen cost matrix may not reflect local healthcare economics. **Binary classification** ignores disease severity levels, and the lack of external validation prevents conclusions about performance in other settings.

Statistically, multiple testing increases family-wise error risk, some assumptions of classical tests are imperfectly met, and small effects may remain undetected.

6.5. Clinical Implications and Applications

The model can support screening by flagging high-risk patients during routine liver function testing. It may also assist in resource allocation by prioritizing specialist referrals and function as a quality assurance tool for identifying potential diagnostic delays.

Implementation requires external validation, threshold calibration, electronic health record integration, and clinician training.

Preliminary economic estimates suggest potential savings of approximately **$12,450 per 100 patients screened**, although formal cost-effectiveness analysis is required.

6.6. Ethical and Fairness Considerations

Detectable gender bias was observed, with an equalized odds difference of **0.085**.

Reweighting reduced this bias by **53.8%** with only a **3.1%** reduction in F2-Score, indicating fairness improvements are feasible without major performance loss.

Ethical deployment requires transparency, accountability for model errors, and continuous monitoring across demographic groups.

Regional specificity also raises equity concerns, reinforcing the need for local validation prior to clinical deployment.

6.7. Conclusions

Cost-sensitive learning with a **5:1 false-negative to false-positive** ratio optimizes liver disease detection by increasing sensitivity by 10.8% and reducing total cost by **27.2%.**

XGBoost provides the best combination of performance, stability, and interpretability, achieving an F2-Score of **0.542** and the lowest coefficient of variation at **2.03%**.

Gender bias is present but can be mitigated using fairness-aware techniques with minimal performance degradation.

Feature importance aligns with clinical knowledge, with total bilirubin and albumin emerging as dominant predictors.

The study shows that training-only transformations prevent leakage, multi-dimensional evaluation improves assessment quality, and statistical rigor strengthens clinical credibility.

For screening applications, sensitivity should be prioritized, stability should accompany performance, and interpretability is essential for clinician trust.

6.8. Future Research Directions

Short-term priorities include external validation, real-world clinical integration studies, and dynamic cost matrix development.

Medium-term goals involve longitudinal modeling, multimodal data integration, and personalized prediction systems.

Implementation-focused research should evaluate workflow impact, economic value, and regulatory pathways.

Technical advances should improve uncertainty quantification, explainability, and continuous learning capabilities.

6.9. Final Recommendations

Researchers should prioritize clinical relevance, comprehensive evaluation, and statistical rigor. Clinicians should adopt cost-sensitive approaches, validate feature importance, and monitor for bias. Healthcare institutions should pilot systems carefully, invest in validation, and establish governance frameworks.

6.10. Concluding Statement

This study shows that machine learning can enhance liver disease detection when aligned with clinical priorities and statistical discipline. Cost-sensitive optimization reflects real medical decision making, where different errors carry different consequences. Effective impact requires not only strong algorithms, but also integration into clinical workflows, ethical oversight, and validation across diverse populations.

REFERENCE

References for Introduction

[1] Asrani, S. K., Devarbhavi, H., Eaton, J., & Kamath, P. S. (2019). Burden of liver diseases in the world. Journal of Hepatology, 70(1), 151-171.
[2] Duseja, A., & Singh, S. P. (2021). Toward a National Program for the Prevention and Control of Liver Diseases in India. Journal of Clinical and Experimental Hepatology.
[3] Kumar, R., Saraswat, V. A., Sharma, B. C., Sakhuja, P., & Sarin, S. K. (2018). Characteristics of liver cirrhosis in India: a systematic review and meta-analysis. The Lancet Gastroenterology & Hepatology.
[4] Tapper, E. B., & Parikh, N. D. (2018). Diagnosis and Management ofCirrhosis and Its Complications: A Review. JAMA, 319(8), 822-835.
[5] Rajkomar, A., Dean, J., & Kohane, I. (2019). Machine Learning in Medicine. New England Journal ofMedicine, 380(14), 1347-1358.

References for Literature Review

[1] Ramana, B. V., Babu, M. S. P., & Venkateswarlu, N. B. (2012). A critical comparative study of liver patients from Andhra Pradesh with other regions in India. International Journal of Computer Science and Informatics, 1(3), 17-22.
[2] Yadav, A., & Sharma, M. (2014). Prediction of liver disease using neural networks. International Journal of Engineering Research and Technology, 3(4), 2278-0181.
[3] Domingos, P. (1999). MetaCost: A general method for making classifiers cost-sensitive. Proceedings of the Fifth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 155-164.
[4] He, H., & Garcia, E. A. (2009). Learning from imbalanced data. IEEE Transactions on Knowledge and Data Engineering, 21(9), 1263-1284.
[5]DeepSeek AI V3 Series

References for Dataset

[1] Source: UCI Machine Learning Repository
[2]Duseja, A., & Singh, S. P. (2021). Toward a National Program for the Prevention and Control of Liver Diseases in India. Journal of Clinical and Experimental Hepatology, 11(5), 523-526.
[3] Eagon, P. K. (2010). Alcoholic liver injury: Influence of gender and hormones. World Journal of Gastroenterology, 16(11), 1377-1384.
[4] Kamath, P. S., & Kim, W. R. (2007). The model for end-stage liver disease (MELD). Hepatology.
[5] Kwo, P. Y., Cohen, S. M., & Lim, J. K. (2017). ACG clinical guideline: evaluation of abnormal liver chemistries. American Journal of Gastroenterology, 112(1), 18-35.

References for Methodology

[1] Wirth, R., & Hipp, J. (2000). CRISP-DM: Towards a standard process model for data mining. Proceedings of the 4th International Conference on the Practical Applications of Knowledge Discovery and Data Mining, 29-39.
[2] Johnson, P. J., et al. (2015). Assessment of liver function in patients with hepatocellular carcinoma: a new evidence-based approach—the ALBI grade. Journal of Clinical Oncology, 33(6), 550-558.
[3] Vickers, A. J., & Elkin, E. B. (2006). Decision curve analysis: a novel method for evaluating prediction models. Medical Decision Making, 26(6), 565-574.

References for Key Findings and Future Work

[1] Tapper, E. B., & Parikh, N. D. (2018). Diagnosis and Management of Cirrhosis and Its Complications: A Review. JAMA, 319(8), 822-835.
[2] Kamath, P. S., & Kim, W. R. (2007). The model for end-stage liver disease (MELD). Hepatology.
[3] Johnson, P. J., et al. (2015). Assessment of liver function in patients with hepatocellular carcinoma: a new evidence-based approach—the ALBI grade. Journal of Clinical Oncology, 33(6), 550-558.
[4] McLean, A. J., & Le Couteur, D. G. (2004). Aging biology and geriatric clinical pharmacology. Pharmacological Reviews, 56(2), 163-184.
[5]DeepSeek AI V3 Series

[6] Norgeot, B., et al. (2020). Minimum information about clinical artificial intelligence modeling: the MI-CLAIM checklist. Nature Medicine, 26(9), 1320-1324.
[7] Vayena, E., Blasimme, A., & Cohen, I. G. (2018). Machine learning in medicine: Addressing ethical challenges. PLoS Medicine, 15(11), e1002689.
[8] Rajkomar, A., Dean, J., & Kohane, I. (2019). Machine learning in medicine. New England Journal of Medicine, 380(14), 1347-1358.
[4] Wiens, J., et al. (2019). Do no harm: a roadmap for responsible machine learning for health care. Nature Medicine, 25(9), 1337-1340.

APPENDIX LaTex: Complete ILPD Cost-Sensitive Learning Project (Python)

```
\usepackage{listings}
\usepackage{xcolor}

\lstdefinestyle{pythonstyle}{
   language=Python,
   basicstyle=\ttfamily\footnotesize,
   numbers=left,
   numberstyle=\tiny,
   stepnumber=1,
   numbersep=8pt,
   frame=single,
   breaklines=true,
   breakatwhitespace=true,
   showstringspaces=false,
   tabsize=2,
   captionpos=b,
   keywordstyle=\color{blue},
   commentstyle=\color{gray},
   stringstyle=\color{teal},
}

\appendix
\section{Python Implementation for Cost-Sensitive Learning on ILPD}
\label{app:ilpd_code}

This appendix presents the complete Python implementation used for the
cost-sensitive classification experiments on the Indian Liver Patient Dataset (ILPD).
The pipeline includes preprocessing, feature engineering, scaling, model training,
baseline evaluation, and cost-sensitive optimisation using XGBoost.

\begin{lstlisting}[style=pythonstyle, caption={ILPD Cost-Sensitive Learning Project
(Python)}]
"""
```

```
ILPD COST-SENSITIVE LEARNING PROJECT
Author: Samuel Adeosun (M66594)
Date: January 3, 2026
Appendix Version (LaTex Format)
"""

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path
import warnings
import sys
from datetime import datetime
from typing import Dict, List, Tuple, Any

warnings.filterwarnings('ignore')
np.random.seed(42)
plt.style.use('seaborn-v0_8-darkgrid')
sns.set_palette('Set2')

# ============= 1. DATA LOADING AND PREPROCESSING ============

def load_and_preprocess_data() -> Tuple[pd.DataFrame, np.ndarray, List[str],
pd.DataFrame]:
    file_paths = [
        "Indian Liver Patient Dataset (ILPD).xlsx",
        Path(__file__).parent / "Indian Liver Patient Dataset (ILPD).xlsx",
        Path(__file__).parent / "data" / "Indian Liver Patient Dataset (ILPD).xlsx",
    ]

    df = None
    for path in file_paths:
        if Path(path).exists():
            df = pd.read_excel(path)
            break

    if df is None:
        raise FileNotFoundError("ILPD dataset file not found.")

    if len(df.columns) == 11:
        df.columns = [
            'Age', 'Gender', 'TB', 'DB', 'Alkphos', 'Sgpt', 'Sgot',
            'TP', 'ALB', 'A/G Ratio', 'Selector'
        ]

    df['Target'] = df['Selector'].apply(lambda x: 1 if x == 1 else 0)
    df = df.drop('Selector', axis=1)

    df['Gender'] = df['Gender'].map({'Male':1,'Female':0,'M':1,'F':0,1:1,0:0}).fillna(0)
```

```
    for col in df.columns:
        if df[col].isnull().any():
            df[col] = df[col].fillna(df[col].median())

    X = df.drop('Target', axis=1)
    y = df['Target'].values

    return X, y, list(X.columns), df


# ===================== 2. FEATURE ENGINEERING =====================

def engineer_features(X: pd.DataFrame, y: np.ndarray) -> Tuple[pd.DataFrame, List[str]]:
    X_eng = X.copy()

    X_eng['AST_ALT_Ratio'] = X_eng['Sgot'] / (X_eng['Sgpt'] + 1e-8)
    X_eng['DB_TB_Ratio'] = X_eng['DB'] / (X_eng['TB'] + 1e-8)
    X_eng['ALP_AST_Ratio'] = X_eng['Alkphos'] / (X_eng['Sgot'] + 1e-8)

    X_eng['Age_TB_Interaction'] = X_eng['Age'] * X_eng['TB']
    X_eng['TB_ALB_Interaction'] = X_eng['TB'] * X_eng['ALB']
    X_eng['Age_ALB_Interaction'] = X_eng['Age'] * X_eng['ALB']

    risk_factors = [
        (X_eng['TB'] > 1.2).astype(int),
        (X_eng['ALB'] < 3.5).astype(int),
        (X_eng['Age'] > 50).astype(int)
    ]

    X_eng['Simple_Liver_Risk_Score'] = sum(risk_factors) / len(risk_factors)
    X_eng['Modified_ALBI'] = 0.66 * np.log10(X_eng['TB'] + 1e-8) - 0.085 * X_eng['ALB']

    return X_eng, list(X_eng.columns)


# ===================== 3. DATA PREPARATION =====================

def prepare_data(X: pd.DataFrame, y: np.ndarray, feature_names: List[str]) -> Dict[str, Any]:
    from sklearn.model_selection import train_test_split
    from sklearn.preprocessing import RobustScaler

    skewed_features = ['TB','DB','Alkphos','Sgpt','Sgot']
    X_log = X.copy()

    for feat in skewed_features:
        if feat in X_log.columns:
            X_log[feat] = np.log1p(X_log[feat])

    scaler = RobustScaler()
```

```
    X_scaled = scaler.fit_transform(X_log)
    X_scaled_df = pd.DataFrame(X_scaled, columns=feature_names)

    X_train, X_test, y_train, y_test = train_test_split(
        X_scaled_df, y, test_size=0.2, random_state=42, stratify=y
    )

    return {
        'X_train':X_train, 'X_test':X_test,
        'y_train':y_train, 'y_test':y_test,
        'feature_names':feature_names, 'scaler':scaler
    }


# ==================== 4. MODEL DEFINITIONS ====================

class CostSensitiveXGBoost:
    def __init__(self, cost_ratio: float = 5.0):
        import xgboost as xgb
        self.model = xgb.XGBClassifier(
            n_estimators=100,
            max_depth=6,
            learning_rate=0.1,
            scale_pos_weight=cost_ratio,
            random_state=42,
            use_label_encoder=False,
            eval_metric='logloss'
        )

    def fit(self, X, y):
        self.model.fit(X,y)
        return self

    def predict(self, X):
        return self.model.predict(X)

    def predict_proba(self, X):
        return self.model.predict_proba(X)


def get_all_models() -> Dict[str, Any]:
    from sklearn.linear_model import LogisticRegression
    from sklearn.ensemble import RandomForestClassifier
    from sklearn.naive_bayes import GaussianNB
    from sklearn.neural_network import MLPClassifier
    import xgboost as xgb

    return {
        'Logistic Regression':
LogisticRegression(class_weight='balanced',max_iter=1000,random_state=42),
```

```
        'Random Forest':
RandomForestClassifier(n_estimators=100,max_depth=10,class_weight='balanced',random_
state=42),
        'XGBoost':
xgb.XGBClassifier(n_estimators=100,max_depth=6,learning_rate=0.1,random_state=42,use
_label_encoder=False,eval_metric='logloss'),
        'Naive Bayes': GaussianNB(),
        'Neural Network':
MLPClassifier(hidden_layer_sizes=(32,16),max_iter=1000,random_state=42)
    }


# ===================== 5. EVALUATION METRICS =====================

class Evaluator:
    def __init__(self, cost_matrix: Dict[str,float]=None):
        self.cost_matrix = cost_matrix or {'TN':0,'FP':1,'FN':5,'TP':0}

    def calculate_all_metrics(self, y_true, y_pred, y_proba=None) -> Dict[str,float]:
        from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
roc_auc_score, confusion_matrix, average_precision_score

        metrics = {}
        metrics['accuracy'] = accuracy_score(y_true,y_pred)
        metrics['precision'] = precision_score(y_true,y_pred,zero_division=0)
        metrics['recall'] = recall_score(y_true,y_pred,zero_division=0)
        metrics['f1'] = f1_score(y_true,y_pred,zero_division=0)

        metrics['f2'] =
(5*metrics['precision']*metrics['recall'])/(4*metrics['precision']+metrics['recall']) if
(metrics['precision']+metrics['recall'])>0 else 0

        tn, fp, fn, tp = confusion_matrix(y_true,y_pred).ravel()
        metrics['specificity'] = tn/(tn+fp) if (tn+fp)>0 else 0

        total_cost = fp*self.cost_matrix['FP'] + fn*self.cost_matrix['FN']
        metrics['total_cost'] = total_cost
        metrics['cost_per_instance'] = total_cost/len(y_true)

        if y_proba is not None:
            metrics['roc_auc'] = roc_auc_score(y_true,y_proba)
            metrics['pr_auc'] = average_precision_score(y_true,y_proba)

        metrics['confusion_matrix'] = {'tn':tn,'fp':fp,'fn':fn,'tp':tp}

        return metrics


# ===================== 6. EXPERIMENTS =====================
```

```
def run_baseline_experiment(data):
    evaluator = Evaluator()
    models = get_all_models()
    results = {}

    for name, model in models.items():
        model.fit(data['X_train'], data['y_train'])
        y_pred = model.predict(data['X_test'])
        y_proba = model.predict_proba(data['X_test'])[:,1]
        results[name] = evaluator.calculate_all_metrics(data['y_test'], y_pred, y_proba)

    return results


def run_cost_sensitivity_experiment(data):
    evaluator = Evaluator()
    results = {}

    for ratio in [1,2,3,5,7,10]:
        model = CostSensitiveXGBoost(cost_ratio=ratio)
        model.fit(data['X_train'], data['y_train'])
        y_pred = model.predict(data['X_test'])
        y_proba = model.predict_proba(data['X_test'])[:,1]
        results[ratio] = evaluator.calculate_all_metrics(data['y_test'], y_pred, y_proba)

    return results


# ===================== 7. MAIN =====================

def main():
    X, y, feature_names, df = load_and_preprocess_data()
    X_eng, all_features = engineer_features(X,y)
    data = prepare_data(X_eng,y,all_features)
    baseline_results = run_baseline_experiment(data)
    cost_results = run_cost_sensitivity_experiment(data)
    return baseline_results, cost_results


if __name__ == "__main__":
    main()
\end{lstlisting}
```